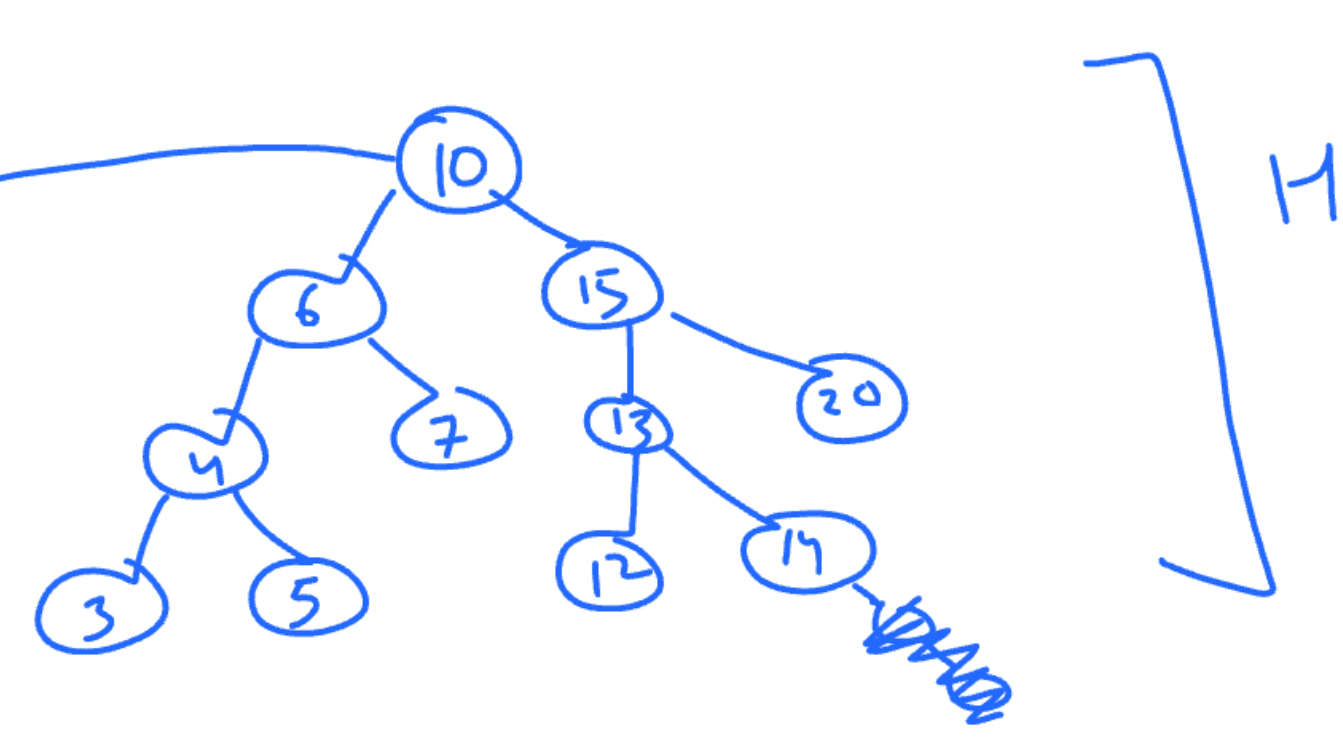
Ordered Maps.

↳ Height-Balanced BSTs.

{Key, value}
{10, value}


10
6
15
4
7
13
20
3
5
12
14

H = $\log_2 n$.

Search: O(H) = O(log n).

Insert: O(H) = O(log n)

Delete: O(H) = O(log n).

↑ BST.

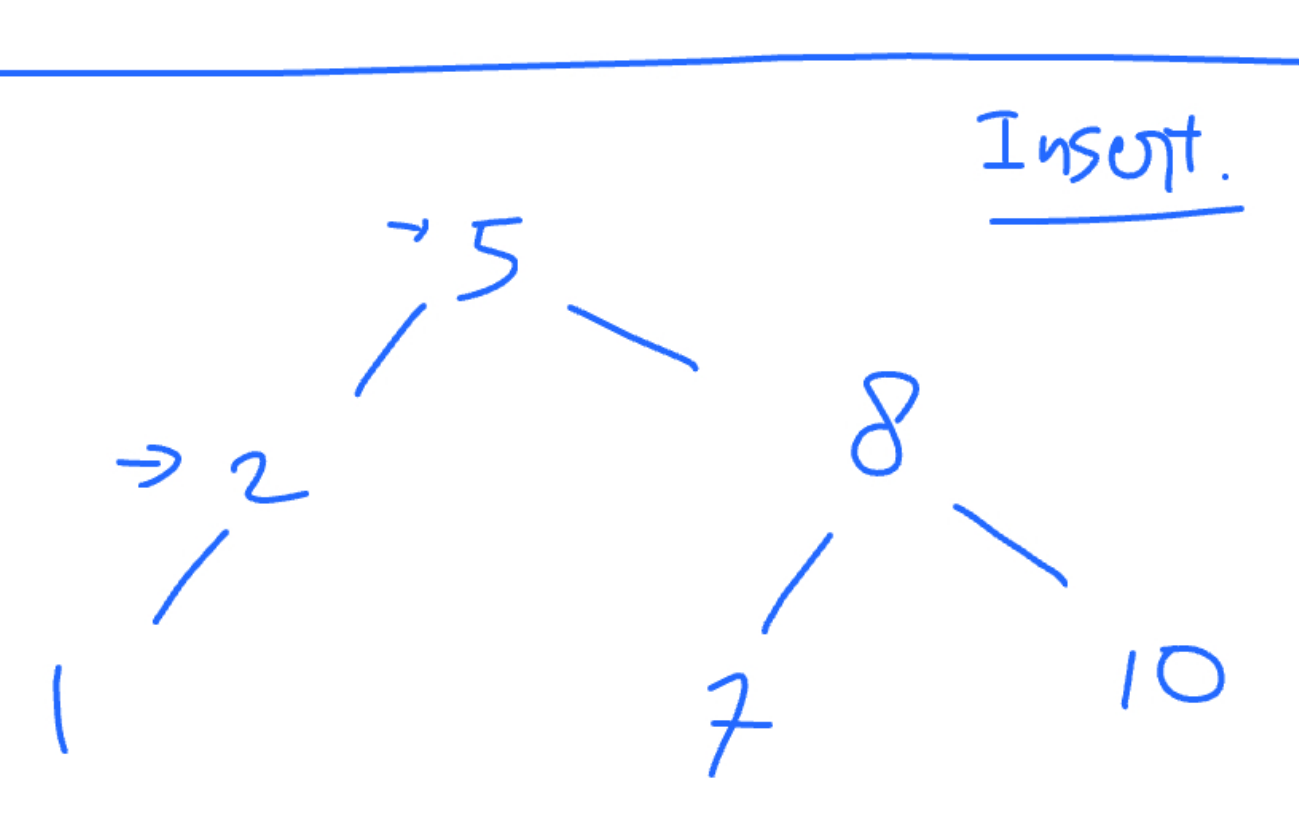
↑ Ht. Balanced BST..

Node {
→ int key;
int value;
Node *left;
Node *right;
}

{K1, V1}
{K2, V2}
⋮
{K5, V5}

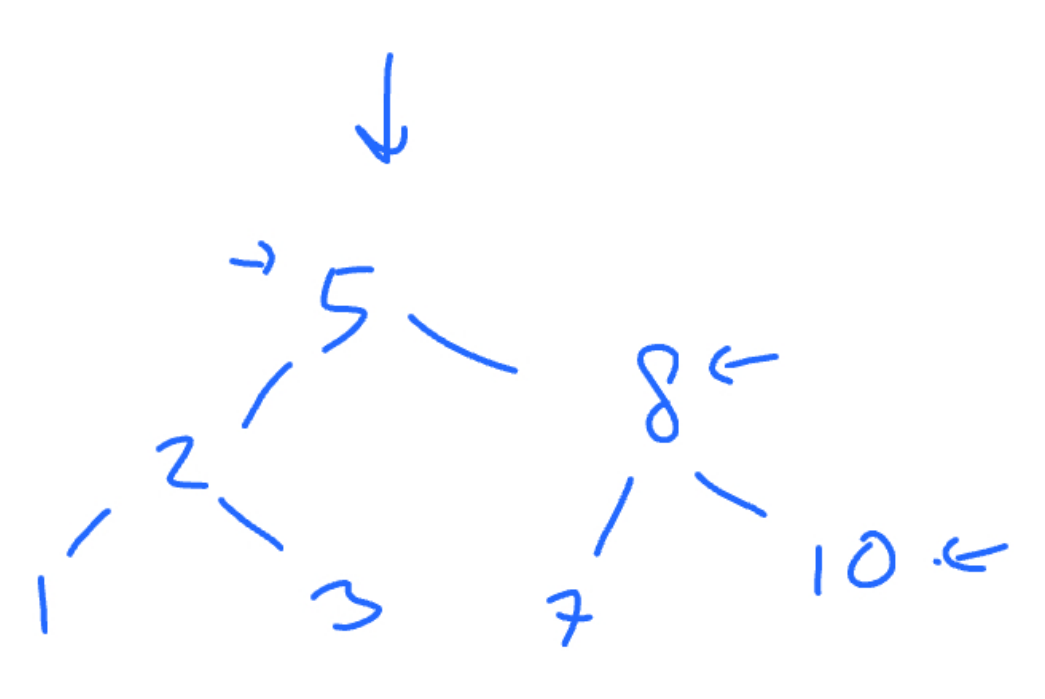
Insert.

val = 3.



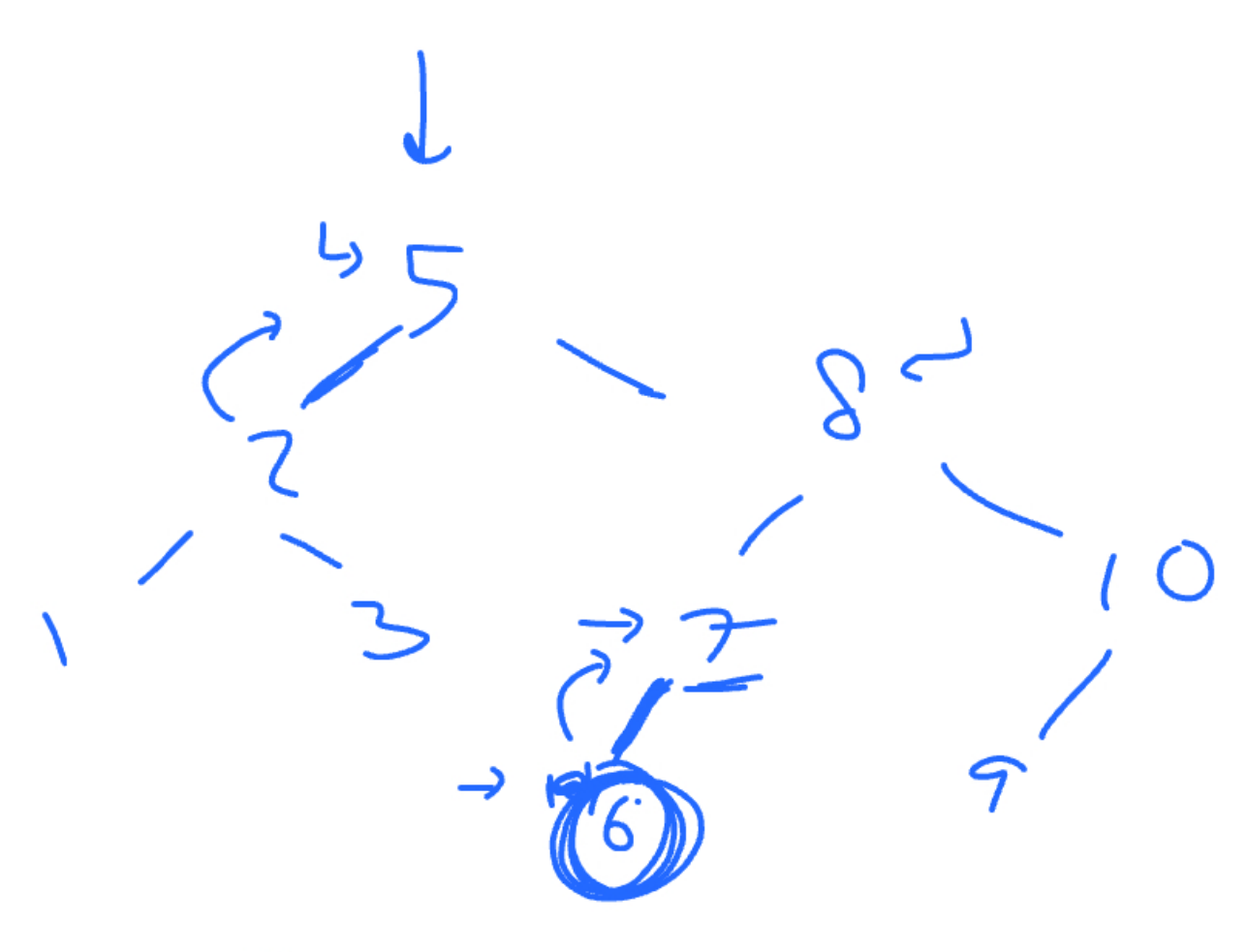
↓

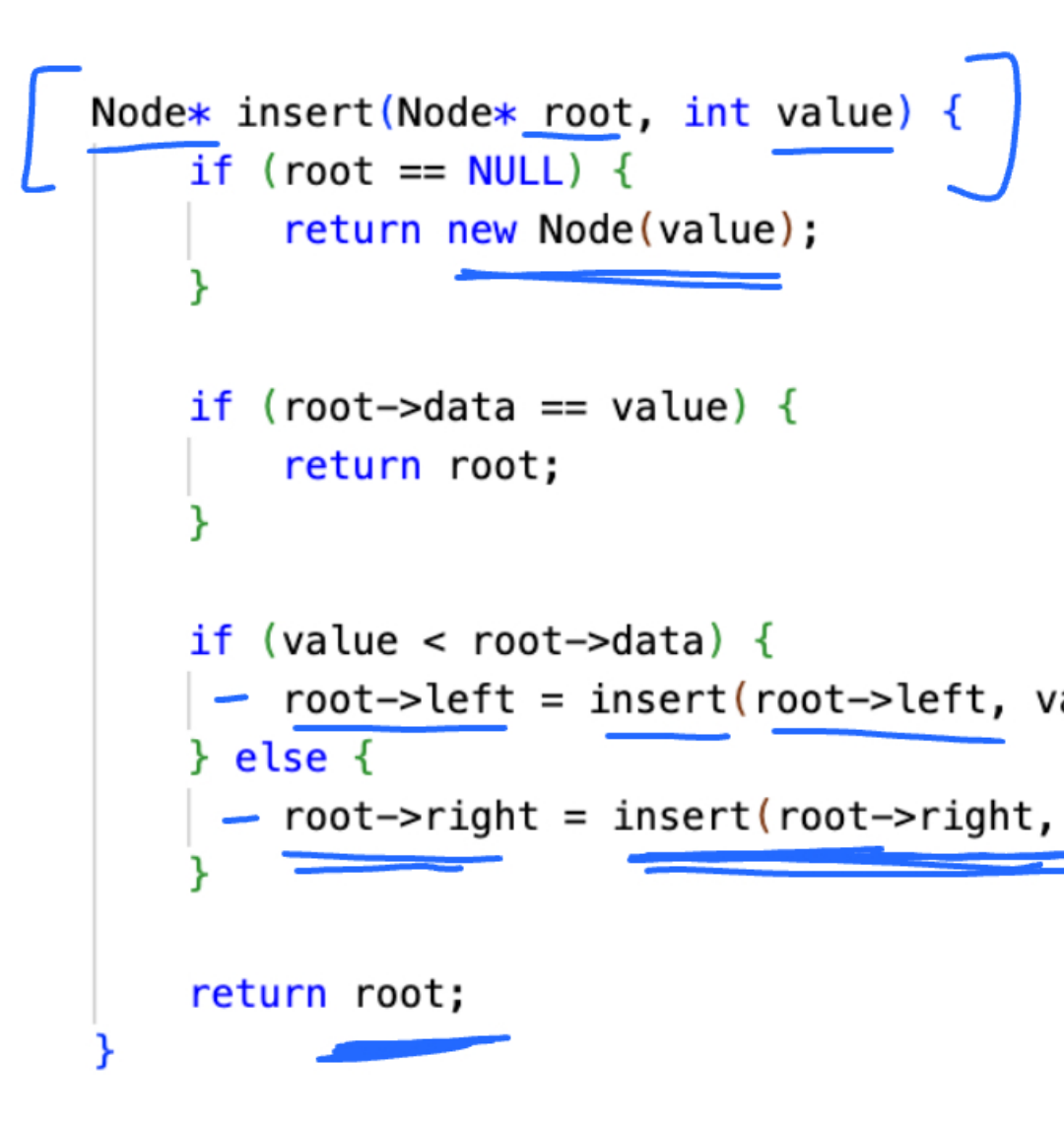
val = 9.



↓

val = 6.


5
2
8
1
3
7
10
9
6

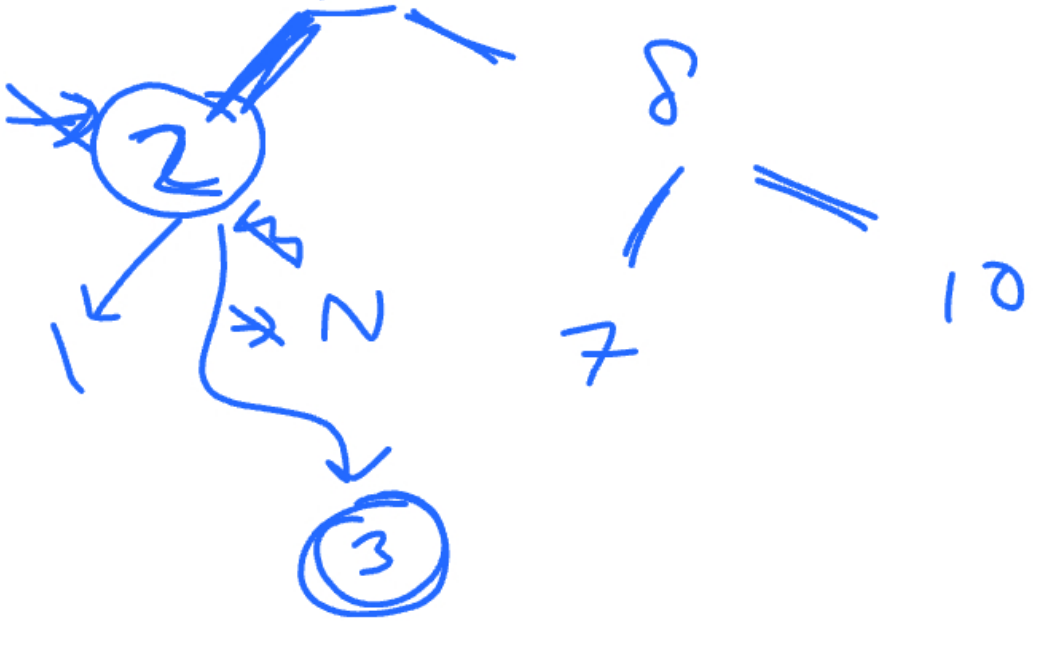


```
Node* insert(Node* root, int value) {
    if (root == NULL) {
        return new Node(value);
    }

    if (root->data == value) {
        return root;
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    } else {
        root->right = insert(root->right, value);
    }

    return root;
}
```

root: Null
↓
root: 6

5.

val = 3.



Null. ③